The class `Grid`

Manual (draft)

Alex Pfaff
nonintersective@gmail.com

December 3, 2025

Contents

Welcome to Sudoku and Congratulations on your acquisition of `Grid`! – it reveals your good taste in sophisticated cool stuff, highlights your fearlessness in the face of structural insanity and combinatorial explosions, and thus betrays your non-standard curiosity towards the unknown (depths of Sudoku geometry); well done!

This manual illustrates the most important features / uses of the `Grid` class; there is also some extensive documentation in the code (docstring, use the Python built-in function `help()` to access the documentation). Feel free to drop me a line (feedback, questions, bug reports, suggestions, comments on Sudoku geometry ...).

Best wishes, Alex

1 Initialize and populate the grid

1.1 Constructor, Insertion

Base number = 3 is the default, tantamount to a classical $(3x3) \times (3x3) = 9 \times 9$ grid

```
>>> g = Grid()   # equivalent to: g = Grid(baseNumber=3)
```

Print object → string-representation: settings of the current grid

```
>>> print(g)
Grid[
  base number     : 3
   -> dimension   : 9    ( = 3 X 3 )
   -> size        : 81   ( = 9 X 9 )
  is initialized  : False
  is alphabetized : False
  is valid        : False
]
```

Pre-existing grids (e.g. numpy.ndarray) or grid sequences (list, tuple, string ...) can be fed into a grid object in two ways: (i) via the `insert` method; via the static constructor `Grid.from_grid(grid)`. The module `PsQ_Grid` provides a handful of predefined grids (`grd1 ... grd5; kgrd1 ... kgrd4; amazing_mod_grd`); for instance:

```
>>> print(grd3)  # encoded as 81-tuple
(3, 7, 8, 4, 2, 6, 5, 9, 1, 2, 5, 4, 1, 8, 9, 6, 3, 7, 1, 9, 6,
7, 3, 5, 4, 2, 8, 5, 6, 9, 3, 7, 8, 2, 1, 4, 7, 3, 2, 5, 1, 4,
8, 6, 9, 4, 8, 1, 9, 6, 2, 7, 5, 3, 9, 2, 5, 8, 4, 3, 1, 7, 6,
8, 1, 3, 6, 5, 7, 9, 4, 2, 6, 4, 7, 2, 9, 1, 3, 8, 5)

>>> g.insert(grd3)

>>> # instead of empty initialization + insert, use class method:
>>> g = Grid.from_grid(grd3)

>>> print(g)
Grid[
  base number     : 3
   -> dimension   : 9    ( = 3 X 3 )
   -> size        : 81   ( = 9 X 9 )
  is initialized  : True
  is alphabetized : False
  is valid        : True
]
```

1.2 Display

There are three ways to display (infos about) the grid: 1. "to-string" method (`print(g)`, see above); 2. grid frame with grid-coordinates; 3. the grid as such. The grid coordinates are: i. running number (1-81, top-left to bottom-right), ii. row number (1-9), iii. column number (1-9), iv. box number (1-9), v. box-column number (1-3), box-row number (1-3), value (1-9 or 0 if not initialized).

```
>>> g.showFrame()  # abbreviated!
run:  1;   row: 1;   col: 1;   box: 1;   box_col: 1;   box_row: 1;   value: 3
run:  2;   row: 1;   col: 2;   box: 1;   box_col: 1;   box_row: 1;   value: 7
run:  3;   row: 1;   col: 3;   box: 1;   box_col: 1;   box_row: 1;   value: 8
run:  4;   row: 1;   col: 4;   box: 2;   box_col: 2;   box_row: 1;   value: 4
run:  5;   row: 1;   col: 5;   box: 2;   box_col: 2;   box_row: 1;   value: 2
run:  6;   row: 1;   col: 6;   box: 2;   box_col: 2;   box_row: 1;   value: 6
run:  7;   row: 1;   col: 7;   box: 3;   box_col: 3;   box_row: 1;   value: 5
run:  8;   row: 1;   col: 8;   box: 3;   box_col: 3;   box_row: 1;   value: 9
run:  9;   row: 1;   col: 9;   box: 3;   box_col: 3;   box_row: 1;   value: 1
run: 10;   row: 2;   col: 1;   box: 1;   box_col: 1;   box_row: 1;   value: 2
. . . .
run: 32;   row: 4;   col: 5;   box: 5;   box_col: 2;   box_row: 2;   value: 7
run: 33;   row: 4;   col: 6;   box: 5;   box_col: 2;   box_row: 2;   value: 8
run: 34;   row: 4;   col: 7;   box: 6;   box_col: 3;   box_row: 2;   value: 2
run: 35;   row: 4;   col: 8;   box: 6;   box_col: 3;   box_row: 2;   value: 1
run: 36;   row: 4;   col: 9;   box: 6;   box_col: 3;   box_row: 2;   value: 4
run: 37;   row: 5;   col: 1;   box: 4;   box_col: 1;   box_row: 2;   value: 7
run: 38;   row: 5;   col: 2;   box: 4;   box_col: 1;   box_row: 2;   value: 3
. . . .
```

```
>>> g.showGrid()

-------------------------
| 3 7 8 | 4 2 6 | 5 9 1 |
| 2 5 4 | 1 8 9 | 6 3 7 |
| 1 9 6 | 7 3 5 | 4 2 8 |
-------------------------
| 5 6 9 | 3 7 8 | 2 1 4 |
| 7 3 2 | 5 1 4 | 8 6 9 |
| 4 8 1 | 9 6 2 | 7 5 3 |
-------------------------
| 9 2 5 | 8 4 3 | 1 7 6 |
| 8 1 3 | 6 5 7 | 9 4 2 |
| 6 4 7 | 2 9 1 | 3 8 5 |
-------------------------
```

1.3 Random Generation

Instead of inserting a pre-existing grid structure, a valid grid can be generated:

```
>>> g.generate_rndGrid()
>>> g.showGrid()

-------------------------
| 3 7 8 | 9 4 1 | 6 5 2 |
| 4 9 2 | 5 7 6 | 1 3 8 |
| 5 6 1 | 3 2 8 | 4 7 9 |
-------------------------
| 6 2 3 | 1 5 4 | 9 8 7 |
| 1 5 7 | 8 6 9 | 3 2 4 |
| 9 8 4 | 7 3 2 | 5 6 1 |
-------------------------
| 2 1 9 | 6 8 5 | 7 4 3 |
| 8 3 5 | 4 1 7 | 2 9 6 |
| 7 4 6 | 2 9 3 | 8 1 5 |
-------------------------
```

As a default, the grid generated is simply a valid Sudoku grid with no (intended!) other special properties, but the generator object allows some specifications (default settings):

```
>>> g.setGenerator(anti_knight=False,
                   modular_knight=False,
                   diagonal_choice=None)
```

When set to `anti_knight=True`, the generator produces a (valid) Sudoku grid that satisfies the **anti knight's move constraint** (cells that are a chess knight's move apart cannot contain the same digit) – on a classical (bounded) grid.

When set to `anti_knight=True, modular_knight=True` (NB: both parameters must be set to `True`), the generator produces a grid that satisfies the anti knight's move constraint on a **toroidal (overflow) grid** (wrap-around structure: when leaving the grid on one side, one re-enters it on the opposite side). The above grid for instance violates this constraint: cell (row=1, col=5; val= 4) now "sees" cell (row=8, col=4; val=4) since they are related via knight's move (2 steps up, 1 step left).

There are three possible settings for `diagonal_choice`: (i) `"main"`: the main diagonal contains 9 distinct digits; (i) `"anti"`: the anti diagonal contains 9 distinct digits; (i) `"both"`: both diagonals contain 9 distinct digits each.

1.4 Checking the Grid

The method `insert`, see Sect. 1.1, has a default setting `automatic_check=True`, which ensures that only valid Sudoku grids are initialized. This option can be disabled simply by setting the parameter to `False`; in that case, the user is responsible for the (in-)validity of the grid.

The "to-string method" displays three properties of the current grid:

```
>>> print(g)
Grid[
  ...
  is initialized  : False
  is alphabetized : False
  is valid        : False
]
```

- `is initialized` is self-explanatory; if `False`, it means that an object has been created empty (`g = Grid()`) and contains zeros as values.
- `is alphabetized`: see Sect. 4 below.
- `is valid`: the current grid is a valid Sudoku grid.

Besides, there are a number of explicit checking methods:

- `g.is_initialized()`: is the current grid initialized?
- `g.is_valid()`: is the current grid a valid Sudoku grid?
- `Grid.is_validGrid(grid)`: corresponding static method (expects a grid argument)
- `g.gridCheckZero()`: checks whether each row, column and box contains maximally one occurrence of each digit (or zero); the difference to `is_valid` is that one or more occurrences of 0 do not count as violation. This method is useful for incomplete grids (construction, solver etc.).
- `g.is_chivalrous(mod: bool = False)`: does the current grid respect the anti knight's move constraint? with the setting `mod = True`: checks the constraint on an overflow grid.
- `g.is_diagonallyDistinct() -> bool, bool`: checks whether either main diagonal or anti diagonal (or both) contain distinct digits.
- `g.is_alphabetized() / g.is_correctly_alphabetized()`: see Sect. 4 below.

2 Basic Getters

- `g.gridRow(r); g.gridCol(c); g.gridBox(b);`
  `g.getBoxRow(br); g.getBoxCol(bc)`
  with $r, c, b \in \{1, 2 ..., 9\}$; $br, bc \in \{1, 2, 3\}$;
  returns row r, col c, box b, boxrow br, boxcolumn bc of the current grid as np.ndarray
- `g.getDiagonal(ax="m")`: returns the (main) diagonal of the current grid as np.ndarray; with `ax="a"`, the anti-diagonal is returned.
- `g.getRelativeBox_c(center)`: returns a 3 $\times$ 3 box around running number `center`. Does not necessarily correspond to an actual grid box.
- `g.getRelativeBox_tl(topLeft)`: returns a 3 $\times$ 3 box with running number `topLeft` as top left cell. Does not necessarily correspond to an actual grid box.
- `g.getRun(r, c)`: returns the running number corresponding to (row=r, col=c).
- `g.gridOut(coord=row)`: returns the current grid as a sequence (81-tuple).
  With `coord=row` (default), the grid is sequentialized row-wise;
  `coord=col`, the grid is sequentialized column-wise;
  `coord=box`, the grid is sequentialized box-wise (row-wise box-internally);
  likewise for `coord=box_row; coord=box_col`.
- `g.grid_toArray(shape)`: returns the current grid as np.ndarray of shape $shape$ (default is 1D); example:

```
>>> g.grid_toArray(shape=(9, 9))

array([[3, 7, 8, 4, 2, 6, 5, 9, 1],
       [2, 5, 4, 1, 8, 9, 6, 3, 7],
       [1, 9, 6, 7, 3, 5, 4, 2, 8],
       [5, 6, 9, 3, 7, 8, 2, 1, 4],
       [7, 3, 2, 5, 1, 4, 8, 6, 9],
       [4, 8, 1, 9, 6, 2, 7, 5, 3],
       [9, 2, 5, 8, 4, 3, 1, 7, 6],
       [8, 1, 3, 6, 5, 7, 9, 4, 2],
       [6, 4, 7, 2, 9, 1, 3, 8, 5]])
```

3 Geometry

Basic operations:

- `g.rotate()`: returns the current grid 90° clockwise.
- `g.diaflect()`: reflects the current grid across the main diagonal ($\Leftrightarrow$ grid^T)

3.1 The (relative) "Phistomefel Field"

One of the most important obvervations in Sudoku geometry: in any given – valid! – Sudoku grid, the "**ring**" (= 16 cells) around the center box contains exactly the same digits as the four 2×2 **corner boxes**. This can be proven. Moreover, on the assumption of a toroidal grid structure, this equivalence can be generalized – or relativized to a specific box; however, with non-center boxes the "ring" is no longer a continuous ring structure, and the corresponding four 2×2 boxes are no longer in the corners (hence, the qualifier *relative*).

The class `Grid` provides three methods to access the components, separately or combined (with $boxrow, boxcolumn \in \{1, 2, 3\}$) as np.ndarrays where non-pertinent digits are represented as zeros:

- `get_relativePhistomefelRing(boxrow, boxcolumn)`
- `get_relativeCornerBoxes(boxrow, boxcolumn)`
- `get_relativePhistomefelField(boxrow, boxcolumn)`

Below, first an illustration of the classical Phistomefel Ring (Field):

```
>>> g.showGrid()                # current grid
    -------------------------
    | 8 6 5 | 9 2 4 | 1 3 7 |
    | 1 7 9 | 6 3 5 | 2 8 4 |
    | 3 2 4 | 8 7 1 | 9 5 6 |
    -------------------------
    | 7 5 6 | 1 8 2 | 4 9 3 |
    | 4 3 8 | 5 9 7 | 6 2 1 |
    | 2 9 1 | 3 4 6 | 8 7 5 |
    -------------------------
    | 5 4 2 | 7 1 9 | 3 6 8 |
    | 6 1 3 | 2 5 8 | 7 4 9 |
    | 9 8 7 | 4 6 3 | 5 1 2 |
    -------------------------

>>> g.get_relativePhistomefelField(2,2)
array([[8, 6, 0, 0, 0, 0, 0, 3, 7],
       [1, 7, 0, 0, 0, 0, 0, 8, 4],
       [0, 0, 4, 8, 7, 1, 9, 0, 0],
       [0, 0, 6, 0, 0, 0, 4, 0, 0],
       [0, 0, 8, 0, 0, 0, 6, 0, 0],
       [0, 0, 1, 0, 0, 0, 8, 0, 0],
       [0, 0, 2, 7, 1, 9, 3, 0, 0],
       [6, 1, 0, 0, 0, 0, 0, 4, 9],
       [9, 8, 0, 0, 0, 0, 0, 1, 2]])
```

```
>>> g.get_relativePhistomefelRing(2,2)
array([[0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 4, 8, 7, 1, 9, 0, 0],
       [0, 0, 6, 0, 0, 0, 4, 0, 0],
       [0, 0, 8, 0, 0, 0, 6, 0, 0],
       [0, 0, 1, 0, 0, 0, 8, 0, 0],
       [0, 0, 2, 7, 1, 9, 3, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0]])


>>> g.get_relativeCornerBoxes(2,2)
array([[8, 6, 0, 0, 0, 0, 0, 3, 7],
       [1, 7, 0, 0, 0, 0, 0, 8, 4],
       [0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0],
       [6, 1, 0, 0, 0, 0, 0, 4, 9],
       [9, 8, 0, 0, 0, 0, 0, 1, 2]])
```

Next, an illustration for a non-center cell:

```
>>> g.get_relativePhistomefelRing(1,1)
array([[0, 0, 0, 9, 0, 0, 0, 0, 7],
       [0, 0, 0, 6, 0, 0, 0, 0, 4],
       [0, 0, 0, 8, 0, 0, 0, 0, 6],
       [7, 5, 6, 1, 0, 0, 0, 0, 3],
       [0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0],
       [9, 8, 7, 4, 0, 0, 0, 0, 2]])

>>> g.get_relativeCornerBoxes(1,1)
array([[0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 9, 7, 6, 2, 0],
       [0, 0, 0, 0, 4, 6, 8, 7, 0],
       [0, 0, 0, 0, 1, 9, 3, 6, 0],
       [0, 0, 0, 0, 5, 8, 7, 4, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0]])
```

See https://github.com/A-Lex-McLee/PseudoQ-2.1/blob/main/Syntax_of_Sudoku_draft.pdf, Sect. 4.3 for further details.

3.2 Triplets

Given a grid g:

```
-------------------------
| 4 9 1 | 5 7 2 | 8 6 3 |
| 8 6 3 | 1 9 4 | 2 5 7 |
| 5 2 7 | 8 6 3 | 4 1 9 |
-------------------------
| 9 5 6 | 2 3 1 | 7 8 4 |
| 3 7 4 | 6 8 5 | 1 9 2 |
| 2 1 8 | 9 4 7 | 6 3 5 |
-------------------------
| 1 4 9 | 3 2 6 | 5 7 8 |
| 7 8 5 | 4 1 9 | 3 2 6 |
| 6 3 2 | 7 5 8 | 9 4 1 |
-------------------------
```

g can be segmented into 27 triplets: segements of box-width proceeding row-wise (4, 9, 1), (5, 7, 2), (8, 6, 3), (8, 6, 3) . . . Will construe triplets as sets, which has two consequences: (i) triplets like (4, 9, 1), (1, 9, 4) and (9, 4, 1) are considered identical, and (ii) the total number of (unique) triplets may be less than 27. Grid g contains the following 14 unique triplets (represented as sorted tuples):

```
((5, 6, 9),
 (4, 7, 8),
 (2, 3, 6),
 (3, 5, 6),
 (5, 6, 8),
 (1, 2, 3),
 (1, 2, 9),
 (5, 7, 8),
 (2, 5, 7),
 (1, 4, 9),
 (3, 6, 8),
 (4, 7, 9),
 (1, 2, 8),
 (3, 4, 7))
```

On this conception, the minimum number of unique triplets is 3 in order to get 9 distinct digits (maximum is 27). It poses an interesting combinatorial conundrum with two variables: the number of triplets for a given (valid) grid, and the constituency of the individual triplets (i.e. the combinatorial distributions of digits across triplets).

For a first impression, consider the frequency distribution (made on the basis of randomly generated grids), illustrated in table 1

no. triplets	occurrences
25	22883329
24	18517142
23	15075995
26	13247241
20	8647525
21	7785875
22	7150343
19	6238520
27	5054871
18	2635381
17	1725354
15	939704
14	638052
16	346101
13	196148
12	67571
11	37500
9	29114
8	9347
7	4508
5	621
3	206
6	172
total: 111,230,620	

Table 1: Occurrences of grids given their numbers of unique triplets

It is not entirely obvious how representative this sample of 111,230,620 grids is, but it seems to suggest a tendency: grids with a smaller number of triplets occur less frequently than grids with a larger number of triplets. All numbers of unique triplets from 3–27 are attested – except for 4 and 10. It can be proven that there cannot be a valid grid constituted by 4 unique triplets. As for 10 triplets – it is not clear yet whether it is simply a severely constrained construction that occurs extremely rarely or whether it is impossible (homework!).

The class `Grid` provides a little "triplet-AP"I.

```
>>> g.get_triplets()        # returns all 27 triplets in order
                            #  (top-left to bottom-right)
>>> g.get_uniqueTriplets() # returns unique triplets
```

```
>>> g.make_tripletGrid()  # every unique triplet is represented by a letter
                          # returns an np.ndarray

array([['a', 'b', 'c'],
       ['c', 'a', 'b'],
       ['b', 'c', 'a'],
       ['d', 'e', 'f'],
       ['g', 'h', 'i'],
       ['j', 'k', 'l'],
       ['a', 'm', 'n'],
       ['n', 'a', 'm'],
       ['m', 'n', 'a']], dtype='<U1')


>>> g.get_tripletDigits() # returns a dictionary indicating
                          # for each digit in which triplet it occurs
                          # (letters according to .make_tripletGrid())

{1: {'a', 'e', 'i', 'j'},
 2: {'b', 'e', 'i', 'j', 'm'},
 3: {'c', 'e', 'g', 'l', 'm'},
 4: {'a', 'f', 'g', 'k'},
 5: {'b', 'd', 'h', 'l', 'n'},
 6: {'c', 'd', 'h', 'l', 'm'},
 7: {'b', 'f', 'g', 'k', 'n'},
 8: {'c', 'f', 'h', 'j', 'n'},
 9: {'a', 'd', 'i', 'k'}}


>>> g.make_tripletGraph()  # returns a complex dictionary -> graph structure
                           # triplets construed as nodes;
                           # edges emerge where intersect(n1, n2) != {}.
                           # Output abbreviated

{'a': {'values': (1, 4, 9),
    'b': {'num_intersect': 0, 'intersection': set()},
    'd': {'num_intersect': 1, 'intersection': {9}},
    'e': {'num_intersect': 1, 'intersection': {1}},
    'i': {'num_intersect': 2, 'intersection': {1, 9}},
     . . .
 'b': {'values': (2, 5, 7),
    'a': {'num_intersect': 0, 'intersection': set()},
    'd': {'num_intersect': 1, 'intersection': {5}},
   . . .
```

Finally, there are two methods to generate minimum triplet grids (i.e. 3 unique

triplets): (i) generate (and insert) a single 3-triplet grid; (ii) generate all(!) 3-triplet grids (as np.ndarray or, optionally, as `GridCollection`, see Sect. 5)).

```
>>> A = (9, 8, 7)
>>> B = (6, 5, 4)
>>> C = (3, 2, 1)

>>> # optionally set a random seed; if insert=False, an array is returned
>>> g.generate_minTripletGrid(A, B, C, seed=None, insert=True)
>>> g.showGrid()

    -------------------------
    | 7 8 9 | 5 4 6 | 3 1 2 |
    | 2 1 3 | 9 7 8 | 6 5 4 |
    | 5 6 4 | 3 2 1 | 7 8 9 |
    -------------------------
    | 9 7 8 | 6 5 4 | 2 3 1 |
    | 1 3 2 | 7 8 9 | 4 6 5 |
    | 4 5 6 | 1 3 2 | 9 7 8 |
    -------------------------
    | 8 9 7 | 4 6 5 | 1 2 3 |
    | 3 2 1 | 8 9 7 | 5 4 6 |
    | 6 4 5 | 2 1 3 | 8 9 7 |
    -------------------------
```

```
>>> triplet_gc = g.get_minTripletCollection(to_gc: bool = False,
                                            abc: bool = False)

>>> triplet_gc.shape
(373248, 9, 9)
```

With the setting `to_gc = True`, a GridCollection object is returned (see Sect. 5); `abc = True` returns a collection of abc-grids (see Sect. 4). Notice that 373,248 is the number of unique distribution; each grid obtained can be recoded in 9! = 362,880 ways such that the actual number of grids with 3 unique triplets is 362,880 $\times$ 373,248 = 135,444,234,240 (for *recoding*, see Sect 4).

4 ABC-Grids and Recoding

Presumably, it is relatively obvious what an alphabetized (valid) Sudoku grid might be (letters instead of digits; no repetitions in rows, columns, boxes). The concept of **abc-grid** as understood here, however, is much more specific. It entails a specific translation procedure where the top row of a (valid) digit grid is mapped to the sequence (A, B, C, D, E, F, G, H, I) yielding a key by which the entire grid is translated. Thus the following grid

```
-------------------------
| 7 8 9 | 5 4 6 | 3 1 2 |
| 2 1 3 | 9 7 8 | 6 5 4 |
| 5 6 4 | 3 2 1 | 7 8 9 |
-------------------------
| 9 7 8 | 6 5 4 | 2 3 1 |
| 1 3 2 | 7 8 9 | 4 6 5 |
| 4 5 6 | 1 3 2 | 9 7 8 |
-------------------------
| 8 9 7 | 4 6 5 | 1 2 3 |
| 3 2 1 | 8 9 7 | 5 4 6 |
| 6 4 5 | 2 1 3 | 8 9 7 |
-------------------------
```

produces the key

{ 7 : ‘A’, 8 : ‘B’, 9 : ‘C’, 5 : ‘D’, 4 : ‘E’, 6 : ‘F’, 3 : ‘G’, 1 : ‘H’, 2 : ‘I’ }

and leads to the corresponding abc-grid

```
-------------------------
| A B C | D E F | G H I |
| I H G | C A B | F D E |
| D F E | G I H | A B C |
-------------------------
| C A B | F D E | I G H |
| H G I | A B C | E F D |
| E D F | H G I | C A B |
-------------------------
| B C A | E F D | H I G |
| G I H | B C A | D E F |
| F E D | I H G | B C A |
-------------------------
```

The alphabetic sequence is rigid, whereas the digit component is flexible. Thus when re-coding the abc-grid, not only can we use the above key in reverse (char to digit), but any valid (top) row may be used as the digit component. This means that the same abc-grid can be mapped to 9! = 362,880 distinct integer grids. In linguistic

parlor, the abc-grid can be seen as the Deep Structure, while the 362880 integer grids are possible Surface Structures (which are all distributionally identical). The totality of valid Sudoku grids can be represented as $\frac{6{,}7\times10^{21}}{362{,}880}$ abc-grids that all have an identical top row.

The set of integer grids that translate to the same abc-grid are referred to here as *horizontal permutation (series)*. The notion of **recoding** is used to refer to a translation from abc-grid to integer grid or integer grid to integer grid within the same horizontal series.

The class `Grid` provides the following functionalities (+ checking methods, see Sect. 1.4):

```
>>> # inserts or returns the abc-grid corresponding to the current grid
>>> g.to_abc_Grid(insert = True)

>>> # returns the corresponding abc-grid as a grid sequence (tuple)
>>> g.canonical_abc_key()

>>> recoder = (1, 2, 3, 4, 5, 6, 7, 8, 9)
>>> g.recode(recoder, insert = True, to_alpha = False)

>>> g.showGrid()

    -------------------------
    | 1 2 3 | 4 5 6 | 7 8 9 |
    | 4 7 5 | 8 3 9 | 1 2 6 |
    | 8 6 9 | 7 2 1 | 4 3 5 |
    -------------------------
    | 2 3 7 | 5 8 4 | 9 6 1 |
    | 9 1 8 | 6 7 2 | 5 4 3 |
    | 5 4 6 | 1 9 3 | 2 7 8 |
    -------------------------
    | 3 8 1 | 9 4 7 | 6 5 2 |
    | 7 9 2 | 3 6 5 | 8 1 4 |
    | 6 5 4 | 2 1 8 | 3 9 7 |
    -------------------------
```

The method `recode` requires a sequence object *recoder* (list, tuple, array, string), which must contain 9 distinct items (digits only or characters only). The method translates $int \rightarrow int; int \rightarrow str; str \rightarrow int; str \rightarrow str$. A string *recoder* requires the setting `to_alpha = True`. This method allows the creation of alphabetized Sudoku grids that are valid, but not "correctly" alphabetized (i.e. not abc-grids):

```
recoder = "algorithm"            # NB: calls .upper() internally!
g.recode(recoder, to_alpha=True)
```

```
g.showGrid()

    -------------------------
    | A L G | O R I | T H M |
    | T H M | A G L | O I R |
    | R O I | T H M | G A L |
    -------------------------
    | G A L | I O R | M T H |
    | H M T | L A G | I R O |
    | O I R | H M T | A L G |
    -------------------------
    | L G A | R I O | H M T |
    | M T H | G L A | R O I |
    | I R O | M T H | L G A |
    -------------------------

>>> g.is_alphabetized()
True

>>> g.is_correctly_alphabetized()
False
```

5 Grid Permutations and Grid Collections

Given a valid grid, if any two columns inside a boxcolumn are swapped, another valid grid is produced resulting in $3! = 6$ possible grid permutations for each boxcolumn; likewise, boxcolumns themselves can be permuted in $3! = 6$ ways. This gives us $3!^4 = 1296$ grid permutations. By geometry, the same reasoning also applies to rows and boxrows; in other words, the manipulation of one grid gives rise to $6^8 = 1296 \times 1296 = 1{,}679{,}616$ valid grids. The set of all these grid permutations is referred to here as *vertical permutation (series)*.

The class `Grid` provides a method to generate the entire horizontal series; by default, this method returns a `GridCollection` object:

```
>>> gc = g.permuteGrids()

>>> print(gc)
GridCollection[
                  internal shape: (1679616, 81)
                  active series:  not_activated
                  . . . . . .
                ]
```

This method has the following signature:

```
permuteGrids(self,
             toCollection: bool = True,
             toDB: bool = False,
             db_name: str = None
```

If set to `toCollection = False`, the collection is returned as np.ndarray, shape (1679616, 81). The method also provides the option to save the collection to an sql database, in which case a database (file) name must be specified `(toDB = True, db_name = "grids_DB")`.

Notice that the grid permutations as describe above do contain rotational symmetry. In other words, when adding all four rotations for each individual grid in a given vertical series, the result is still 1,679,616 distinct grids. Reflection, on the other hand, does produce new grids not yet contained in the collection. Thus when adding g^T for each grid g in a vertical series, we have $2 \times$ 1,679,616 $=$ 3,359,232 distinct (valid) grids. There is a method to obtain this larger collection:

```
permuteGrids_T(self,
               toCollection: bool = True,
               toDB: bool = False,
               db_name: str = None)
```

There are other ways to create a `GridCollection` object; one example was already hinted at in Sect. 3.2:

```
>>> triplet_gc = g.get_minTripletCollection(to_gc: bool = True)

>>> print(triplet_gc)
GridCollection[
                internal shape: (373248, 81)
                active series: not_activated
               . . . . . .
```

`GridCollection` objects can be used to examine patterns grids in larger contexts or as datasets for deep learning projects. See the documentation *GridCollection_class_manual.pdf*.

6 Appendix: The BASENUMBER

At the outset, the idea was to have a framework for generalized Sudoku grids of variable sizes; "generalized Sudoku grid" means here that the grid (size) is a double-quadratic expansion of the base number:

base number: n	**box dim:** n^1	**grid dim:** n^2	**number cells:** n^4
1	1×1	1×1	1
2	2×2	4×4	16
3 $\Leftarrow$	$\mathbf{3} \times \mathbf{3}$	$\mathbf{9} \times \mathbf{9}$	**81** (classical)
4	4×4	16×16	256
5	5×5	25×25	625

The base number is the numer of columns/rows per box, and the number of box-columns/boxrows per grid; (base number)^2 is the number of columns/rows per grid; (base number)^4 is the number of cells in the grid. Moreover, the base number is required for various algorithms on the grid structure. A classical 9×9 Sudoku grid has base number 3; therefore, the default setting is `Grid(baseNumber = 3)`.

Currently, the basenumber is largely an experimental feature of the `Grid` class, and for many functionalities it is not recommended to choose a base number > 4 (in fact, some methods presuppose the default setting). Consider some non-classical settings:

```
>>> g = Grid(1)            # boring! only one possibility
>>> g.generate_rndGrid()
>>> g.showGrid()

    -----
    | 1 |
    -----


>>> g = Grid(2)           # potentially useful for toy examples
>>> g.generate_rndGrid()
>>> g.showGrid()

    -------------
    | 1 3 | 4 2 |
    | 4 2 | 1 3 |
    -------------
    | 2 4 | 3 1 |
    | 3 1 | 2 4 |
    -------------
```

```
>>> g = Grid(4)
>>> g.generate_rndGrid()
>>> g.showGrid()

    ---------------------------------------
    | 8 9 13 14 | 7 16 15 1 | 2 5 6 3 | 4 11 10 12 |
    | 2 10 16 7 | 13 11 5 4 | 9 14 12 8 | 1 3 6 15 |
    | 11 15 6 12 | 9 8 3 10 | 13 7 1 4 | 5 16 14 2 |
    | 1 4 3 5 | 14 2 6 12 | 15 16 10 11 | 8 7 13 9 |
    ---------------------------------------
    | 7 16 11 2 | 1 14 10 6 | 4 3 15 5 | 9 8 12 13 |
    | 15 13 4 9 | 2 5 12 3 | 8 6 11 7 | 14 10 1 16 |
    | 12 3 8 6 | 15 13 9 16 | 1 10 2 14 | 7 5 11 4 |
    | 14 1 5 10 | 8 4 11 7 | 12 13 16 9 | 3 2 15 6 |
    ---------------------------------------
    | 6 11 14 8 | 10 1 2 13 | 16 4 5 15 | 12 9 7 3 |
    | 10 12 9 1 | 11 3 16 14 | 6 8 7 2 | 15 13 4 5 |
    | 13 5 15 4 | 6 12 7 8 | 3 1 9 10 | 16 14 2 11 |
    | 16 7 2 3 | 5 9 4 15 | 14 11 13 12 | 6 1 8 10 |
    ---------------------------------------
    | 3 2 12 16 | 4 6 14 11 | 7 9 8 13 | 10 15 5 1 |
    | 5 6 7 11 | 3 15 13 9 | 10 12 14 1 | 2 4 16 8 |
    | 9 8 10 13 | 16 7 1 2 | 5 15 4 6 | 11 12 3 14 |
    | 4 14 1 15 | 12 10 8 5 | 11 2 3 16 | 13 6 9 7 |
    ---------------------------------------
```

Other than the messy formatting for 2-digit numbers (`showGrid`), the real problem stems from the complexity of the grid structure, which easily leads to a combinatorial explosion. Consider the formula to calculate the number of possible grid permutations $|P(\mathbb{G})|$ ($\rightarrow$ vertical series; see Sect. 5):

$$|P(\mathbb{G}_{basenumber})| = (basenumber!)^{exp} \qquad \text{with } exp = 2 * basenumber + 2$$

$$|P(\mathbb{G}_1|) = 1!^4 = 1$$

$$|P(\mathbb{G}_2)| = 2!^6 = 64$$

$$|P(\mathbb{G}_3)| = 3!^8 = 1679616$$

$$|P(\mathbb{G}_4)| = 4!^{10} = 63403380965376$$

$$|P(\mathbb{G}_5)| = 5!^{12} = 8916100448256000000000000$$

This number grows super-exponentially, and already for base number 4, it is infeasible to attempt to generate all grid permutations (at least on computers that only

manage household runtimes). Similar problems are encountered with other algorithms, e.g. `generate_rndGrid`, let alone the issues faced by a `GridCollection` object that attempting to manage billions of grids.

In short, feel free to experiment, but keep in mind that runtime easily runs wild! When in doubt, stick to the default setting base number = 3!